

Split 33 — Motor central de nómina base

Objetivo

Convertir la nómina en un motor central de cálculo, no en una pantalla aislada. El cálculo agregado existente se mantiene por compatibilidad, pero se añade una capa profesional de conceptos que permite explicar, imprimir, auditar e integrar incidencias.

Fase 1 — Motor de nómina base

Pasos

1. Mantener el cálculo ordinario mensual estable: salario base, complementos, antigüedad, incentivos, prorata de pagas extra, bases, deducciones, coste empresa, IRPF y neto.
2. Añadir una salida canónica de líneas de concepto a partir del resultado agregado.
3. Añadir totales derivados por conceptos: devengos, deducciones, informativos, base cotizable, base tributable y neto por conceptos.
4. Cubrir con tests una nómina ordinaria y una nómina afectada por IT.

Implementado

- `backend/app/services/payroll_concept_engine.py`
- `calculate_payroll_engine_result()` ahora devuelve:
 - `concept_lines`
 - `concept_totals`

Fase 2 — Estructura profesional de conceptos

Pasos

1. Ampliar `PayrollConcept` para recoger campos profesionales:
 - código
 - nombre
 - tipo
 - naturaleza
 - cotiza
 - tributa
 - afecta a bruto
 - afecta a neto
 - origen
 - orden de impresión
 - fórmula
2. Exponer esos campos por schema/API.

3. Añadir migración ligera conservadora para entornos sin Alembic.
4. Persistir líneas automáticas del motor con `source_key` propio, sin destruir líneas manuales ni líneas específicas de incidencias.

Implementado

- Campos nuevos en `PayrollConcept`:
 - `affects_gross`
 - `affects_net`
 - `formula`
- `PayrollItemResponse` expone banderas relevantes para UI/recibo.
- `backend/app/services/payroll_concept_items.py` sincroniza líneas del motor con prefijo `ENGINE:{payroll_id}:`...

Fase 3 — Recibo de nómina

Pasos

1. Crear un endpoint específico de recibo profesional.
2. Separar bloques:
 - cabecera empresa
 - datos trabajador
 - periodo
 - devengos
 - deducciones
 - bases
 - líquido
 - coste empresa
 - pie legal/simulado
3. Crear una vista frontend densa tipo ERP.
4. Añadir exportación imprimible a PDF desde navegador sin dependencias pesadas.

Implementado

- Endpoint de datos:
 - `GET /payrolls/{payroll_id}/receipt`
- Endpoint imprimible:
 - `GET /payrolls/{payroll_id}/receipt/print`
- Schema:
 - `backend/app/schemas/payroll_receipt.py`
- Servicio:
 - `backend/app/services/payroll_receipt.py`
 - `backend/app/services/payroll_receipt_print.py`
- Frontend:
 - `frontend/src/components/payrolls/PayrollReceiptModal.jsx`
 - acción Recibo en `PayrollTable.jsx`

- botón Exportar PDF en la modal del recibo

Fase 4 — Integración total con incidencias

Pasos

1. Mantener la segmentación de incidencias como motor especializado.
2. Procesar incidencias sobre nóminas mensuales.
3. Aplicar cambios al resultado agregado de la nómina.
4. Regenerar líneas canónicas de concepto tras el procesamiento de incidencias.
5. Mantener snapshots y control de versión.
6. Mostrar explicación didáctica de incidencias dentro del recibo.
7. Crear casos demo cerrados para validar la explicación en contexto comercial.
8. Explicar por qué cambian las bases de cotización e IRPF cuando hay incidencias.
9. Explicar cada línea del recibo para uso docente.

Implementado

- `incident_payroll_orchestrator` sincroniza las líneas canónicas tras aplicar incidencias.
- `payroll_application_service` sincroniza líneas canónicas tras crear, actualizar o preparar nóminas.
- El recibo expone:
 - `incident_summary`
 - `incident_explanations`
 - `base_explanations`
 - `line_explanations`
- La modal de recibo muestra:
 - bloque de lectura didáctica de incidencias
 - bloque de bases y cotización
 - bloque de explicación línea por línea

Subfase Split 33 — Recibo didáctico de incidencias

Objetivo

Evitar que la integración con incidencias sea solo una suma de importes. El alumno debe ver por qué una incidencia modifica salario, prestación, complemento, descuento, bases y líquido.

Implementado

- Resumen global de incidencias en el recibo:
 - días de incidencia
 - días de IT
 - días no cotizados
 - total de prestaciones
 - total de complementos empresa

- total de descuentos
- efecto neto mostrado por los segmentos
- Explicación por segmento:
 - tipo de incidencia
 - periodo afectado
 - días naturales
 - días de nómina
 - salario del tramo
 - prestación
 - complemento
 - descuento
 - tratamiento de cotización
 - conceptos relacionados
 - puntos de aprendizaje
- Vista frontend:
 - panel LECTURA DIDÁCTICA
 - tarjetas por segmento
 - chips de conceptos relacionados

Archivos tocados

- backend/app/services/payroll_receipt.py
- backend/app/schemas/payroll_receipt.py
- frontend/src/components/payrolls/PayrollReceiptModal.jsx
- backend/tests/test_payroll_receipt.py

Subfase Split 33 — Casos demo de nómina con incidencias

Objetivo

Disponer de datos demo realistas para que la pantalla de recibo no se enseñe vacía ni con importes abstractos. La demo debe permitir abrir el histórico de nóminas, pulsar **Recibo** y explicar casos docentes sin preparar datos manualmente.

Casos incorporados

1. **Javier Romero Sánchez — IT común de 8 días**
 - Periodo: 06/05/2026 a 13/05/2026.
 - Días trabajados: 22.
 - Días IT: 8.
 - Salario base ordinario: 1.450,00 €.
 - Salario trabajado: 1.063,33 €.
 - Prestación IT: 232,00 €.
 - Complemento empresa IT: 154,67 €.
 - Bruto resultante: 1.450,00 €.
 - Uso docente: sustitución de salario por prestación y complemento.
2. **Carmen López Torres — Recaída IT de 7 días**

- Periodo: 12/05/2026 a 18/05/2026.
- Días trabajados: 23.
- Días IT: 7.
- Salario base ordinario: 1.825,00 €.
- Complemento salarial: 120,00 €.
- Salario trabajado: 1.399,17 €.
- Prestación IT: 255,50 €.
- Complemento empresa IT: 170,33 €.
- Bruto resultante: 1.945,00 €.
- Uso docente: continuidad/recaída y trazabilidad del tramo.

3. Ana Pérez Navarro — Ausencia no retribuida de 1 día

- Fecha: 27/05/2026.
- Días trabajados: 29.
- Días no cotizados: 1.
- Salario base ordinario: 1.510,00 €.
- Complemento salarial: 40,00 €.
- Salario trabajado: 1.459,67 €.
- Descuento didáctico del segmento: 50,33 €.
- Bruto resultante: 1.499,67 €.
- Uso docente: diferencia entre ausencia sin prestación y situaciones protegidas.

Implementado

- Nuevo helper:
 - `backend/app/seed_demo_payroll_incident_cases.py`
- Integración en seed principal:
 - `backend/app/seed_demo.py`
- El seed sincroniza líneas automáticas del motor para las nóminas afectadas.
- Los recibos demo quedan listos para abrirse desde el histórico sin intervención manual.

Subfase Split 33 — Explicación didáctica de bases

Objetivo

Explicar al alumno que una nómina no termina en bruto/neto. Las incidencias también pueden afectar días cotizados, bases de Seguridad Social, base de IRPF y coste empresarial.

Implementado

- El recibo expone `base_explanations` con explicación de:
 - `BASE_CC`
 - `BASE_CP`
 - `BASE_DESEMPLEO_FORMACION_FOGASA`
 - `BASE_IRPF`
- Cada explicación incluye:
 - código
 - título

- importe
- fórmula descriptiva
- marca de si está afectada por incidencia
- explicación textual
- puntos de aprendizaje
- La modal de recibo incluye panel BASES Y COTIZACIÓN.
- El panel explica:
 - diferencia entre días trabajados y días cotizados
 - efecto de IT con prestación/complemento
 - efecto de ausencia no retribuida con días no cotizados
 - diferencia entre base de cotización y base IRPF

Archivos tocados

- backend/app/services/payroll_receipt.py
- backend/app/schemas/payroll_receipt.py
- frontend/src/components/payrolls/PayrollReceiptModal.jsx
- backend/tests/test_payroll_receipt.py

Subfase Split 33 — Explicación línea por línea

Objetivo

Convertir el recibo en una herramienta de aprendizaje, no solo en una reproducción visual. Cada concepto debe poder explicarse a un alumno sin salir de la pantalla.

Implementado

- El recibo expone `line_explanations`.
- Cada línea explica:
 - código
 - nombre
 - sección del recibo: devengo, deducción, base informativa, coste empresa o informativo
 - importe
 - tipo de concepto
 - categoría
 - origen
 - si afecta al bruto
 - si afecta al neto
 - si cotiza
 - si tributa
 - fórmula, si existe
 - explicación textual
 - puntos de aprendizaje
- La modal añade panel LECTURA LÍNEA POR LÍNEA.

- El panel permite enseñar por qué una línea suma, resta, informa una base o representa coste empresa.
- El panel marca visualmente conceptos que cotizan, tributan, afectan a bruto/neto o proceden de incidencias.

Archivos tocados

- `backend/app/services/payroll_receipt.py`
- `backend/app/schemas/payroll_receipt.py`
- `frontend/src/components/payrolls/PayrollReceiptModal.jsx`
- `backend/tests/test_payroll_receipt.py`

Subfase Split 33 — Exportación imprimible del recibo

Objetivo

Permitir que el recibo simulado pueda entregarse, imprimirse o guardarse como PDF desde el navegador sin introducir dependencias pesadas de generación PDF en backend.

Decisión técnica

No se añade todavía un motor PDF binario como WeasyPrint, Playwright, ReportLab o Chromium. La fase implementa una vista HTML autónoma, con CSS de impresión A4 y botón **Imprimir / Guardar como PDF**. Esto mantiene la demo ligera, portable y compatible con el enfoque open source/Fedora.

Implementado

- Nuevo endpoint:
 - `GET /payrolls/{payroll_id}/receipt/print`
- Nuevo servicio:
 - `backend/app/services/payroll_receipt_print.py`
- La vista imprimible incluye:
 - cabecera del recibo
 - empresa, centro y trabajador
 - periodo y métricas principales
 - devengos
 - deducciones
 - bases
 - coste empresa
 - explicaciones didácticas de bases
 - explicaciones de incidencias
 - explicación línea por línea
 - pie legal de simulación
- Frontend:
 - `frontend/src/services/payrollApi.js` añade `buildPayrollReceiptPrintUrl()`

- `PayrollReceiptModal.jsx` añade botón Exportar PDF
- El botón abre una nueva pestaña con la vista preparada para imprimir o guardar como PDF.

Limitación asumida

La exportación PDF depende del diálogo de impresión del navegador. No se genera todavía un fichero PDF binario desde el servidor.

Archivos tocados

- `backend/app/services/payroll_receipt_print.py`
- `backend/app/payroll_salary_structure_routes.py`
- `frontend/src/services/payrollApi.js`
- `frontend/src/components/payrolls/PayrollReceiptModal.jsx`
- `backend/tests/test_payroll_receipt_print.py`

Fase 5 — Regularizaciones

Pasos previstos

1. Preparar conceptos de origen `REGULARIZACION`.
2. Calcular diferencias contra snapshots previos.
3. Generar línea de atraso o descuento.
4. Trazar origen: incidencia tardía, baja recibida tarde, cambio salarial retroactivo, antigüedad, convenio o recálculo.
5. Bloquear reescritura de nóminas cerradas y derivar a regularización.

Preparación técnica completada

La estructura por conceptos ya incluye origen, fórmula, afectación a bruto/neto, base cotizable y base tributable, por lo que las regularizaciones podrán expresarse como líneas económicas trazables.

Tests añadidos

- `backend/tests/test_payroll_concept_engine.py`
- `backend/tests/test_payroll_receipt.py`
- `backend/tests/test_seed_demo_payroll_incident_cases.py`
- `backend/tests/test_payroll_receipt_print.py`

Cobertura principal:

- nómina ordinaria completa
- salario reducido por incidencia/IT
- construcción de conceptos desde objeto `Payroll`
- agrupación de líneas de recibo
- totales agregados frente a totales por conceptos
- explicación didáctica de segmentos de incidencia
- resumen económico de incidencias
- importes de casos demo IT/recaída/ausencia

- explicación didáctica de bases ordinarias
- explicación de bases con IT
- explicación de bases con ausencia y días no cotizados
- explicación línea por línea de devengos, deducciones y bases
- explicación de origen por incidencia
- explicación de conceptos no cotizables/no tributables
- render HTML imprimible del recibo
- escapado de contenido en la vista imprimible
- nombre de archivo seguro para exportación

Pendiente inmediato recomendado

1. Regularizaciones como módulo separado, no antes de cerrar recibo e integración visual con incidencias.
2. Generación PDF binaria en servidor si un piloto lo exige.